Vector Databases & RAG

1. What is a Vector Database?

A **Vector Database** is a specialized database designed to store, index, and query **high-dimensional vector embeddings**. Unlike traditional databases that store exact values (numbers, strings, rows), vector databases store mathematical representations of data so that we can perform **similarity search** based on meaning rather than exact matches.

Core Ideas

- **High-dimensional embeddings** — Text, images, audio or video are converted into dense numerical vectors (typically 384–3072 dimensions) that capture semantic meaning.
- **Semantic search** — Queries return results that are conceptually related, not just keyword matches.
- **Unstructured data friendly** — Handles free-form text, images, documents, code, and multi-modal data that traditional RDBMS struggle with.

Why embeddings matter

An embedding model (e.g., OpenAI `text-embedding-3-small/large`, or open-source models) maps a piece of text into a point in a high-dimensional space. Texts with similar meaning end up close to each other in that space. The vector database's job is to store millions or billions of these points and quickly find the nearest neighbours to a query vector.

> **Key Takeaway**
> Vector DBs turn "search by meaning" into a fast, scalable database operation. They are the memory layer that modern LLMs and AI agents rely on.

---

2. Why Do We Need Vector Databases?

Traditional keyword search (BM25, SQL `LIKE`, Elasticsearch keyword mode) fails when the user's intent is expressed with different words than those present in the documents.

Failed Keyword Search vs Successful Semantic Search

Example query: *"How to fly to Mars?"*

- **Keyword search** returns irrelevant results such as "Best Coffee in London", "Basic Carpentry Skills", or "Cooking with Spices" simply because those documents do not contain the exact tokens.
- **Semantic search** understands the intent and returns highly relevant documents: "A Roadmap to Mars Colony", "Current Rocket Propulsion for Interplanetary Travel", "The Human Journey to the Red Planet".

Two Critical Problems Solved

1. **Overcomes the semantic gap** — The gap between how humans express ideas and how computers traditionally match text.
2. **Gives LLMs long-term memory** — An LLM's context window is limited. A vector database acts as external, searchable memory so the model can retrieve relevant facts on demand (the foundation of RAG).

> **Key Takeaway**
> Without vector databases, LLMs are limited to their training cutoff and the short context you can fit in a prompt. With them, they can ground answers in private, up-to-date knowledge.

---

3. How Vector Databases Work

The pipeline is conceptually simple but engineered for extreme speed and scale:

Step-by-step flow

1. Text → Embedding Model
Raw text (documents or a user query) is passed through an embedding model (OpenAI, Cohere, Voyage, or open-source such as all-MiniLM, BGE, E5). The model outputs a fixed-size dense vector.

2. Vector Storage
The resulting vectors are stored in the vector database along with optional metadata (source file, page number, timestamp, author, tags, etc.).

3. Similarity Search
When a query arrives, it is also embedded. The database then finds the vectors that are closest to the query vector using one of several distance metrics.

Common Similarity Metrics

- **Cosine Similarity** — Measures the angle between two vectors. Range [−1, 1]; higher is more similar. Preferred for most text embeddings because magnitude is often less important than direction.
- **Euclidean Distance (L2)** — Straight-line distance. Smaller distance = more similar. Sensitive to vector magnitude.
- **Dot Product** — Related to cosine when vectors are normalized. Fast to compute; used by many production systems when embeddings are unit-length.

Formula (Cosine Similarity):

```
cos(θ) = (A · B) / (||A|| × ||B||)
```

> **Key Takeaway**
> Everything starts and ends with the quality of the embedding model. A better embedding → better retrieval → better RAG answers.

---

4. Vector Database vs Traditional Database

Comparison Point	Traditional Database (SQL)	Vector Database
Data Type	Structured data: tables, schemas, numbers, exact strings	Unstructured / semi-structured: embeddings of text, images, audio, video
Search Method	Exact keyword / predicate matching ("Mars" only matches "Mars")	Semantic similarity search ("How to fly to Mars" finds related concepts)
Query Processing	Row-by-row or index scans; sequential or limited parallel	Highly parallel vector arithmetic (ANN algorithms)
Storage Format	Flat tables & typed columns defined by schema	High-dimensional dense vectors + optional metadata filters
Speed & Scalability	Excellent for simple reads & transactions; scales vertically	Blazing fast for nearest-neighbour search; scales horizontally across clusters

> **Practical note:** In real systems you almost always use both. A traditional database (or metadata store) holds structured attributes; the vector database holds the embeddings and supports semantic retrieval. Hybrid search (keyword + vector) is increasingly common.

---

5. Key Components of a Vector Database

1. **Embeddings**
   Convert unstructured data (text, images…) into dense numerical vectors. This step is usually

performed outside the database by an embedding model, then the vectors are ingested.

2. **Indexing (HNSW & others)**
   Build efficient data structures so that approximate nearest-neighbour (ANN) search is fast even with millions of vectors. HNSW (Hierarchical Navigable Small World) is currently the most popular graph-based index. Other options include IVF, PQ, ScaNN, DiskANN.

3. **Storage**
   Persist the high-dimensional vectors and their associations (IDs, metadata, original text chunks). Some systems keep everything in memory; others spill to disk while keeping an in-memory index.

4. **Metadata & Filtering**
   Attach scalar metadata (source, date, category, access control tags) and apply filters before or after the vector search. This is critical for multi-tenant and enterprise use cases.

5. **Query Engine**
   Accepts a query vector (or text that is embedded on the fly), runs the similarity search, applies filters, and returns the top-k most similar items together with their scores and metadata.

> **Key Takeaway**
> HNSW indexing + good metadata filtering is what makes production vector search both fast and useful. Always design your chunking + metadata strategy carefully — retrieval quality depends on it.

---

6. Popular Vector Databases (2026 Landscape)

Database	Strength
Chroma	AI-native open-source embedding database. Extremely easy to start with — perfect for prototypes, notebooks, and small-to-medium applications. Great developer experience and LangChain/LlamaIndex integration.
Pinecone	Fully managed, serverless vector database. Excellent for production apps that need effortless scaling, high availability, and minimal ops.
Weaviate	Flexible, cloud-native open-source database that supports both structured and unstructured data. Strong hybrid search (vector + BM25), GraphQL API, and multi-modal capabilities.
Qdrant	Lightning-fast, high-performance vector database written in Rust. Excellent filtering, payload support, and efficiency.
Milvus	Mature, cloud-native open-source vector database built for massive scale. Used by many large enterprises. Supports GPU acceleration.
PGVector	Open-source extension that adds vector search capability directly inside PostgreSQL. Ideal when you already run Postgres.

> **Recommendation for this course:** We start with **Chroma** because it requires almost zero setup, runs locally or in-memory, and integrates cleanly with OpenAI embeddings and LangChain. Once you understand the patterns, migrating to Pinecone, Qdrant or PGVector is straightforward.

---

7. Retrieval Augmented Generation (RAG)

RAG is the dominant pattern for giving LLMs access to private or up-to-date knowledge without fine-tuning. It combines retrieval from a knowledge base with generation by an LLM.

The Classic RAG Pipeline (5 stages)

1. **User Query**
   The user asks a question or gives an instruction in natural language.

2. **Embedding**
   The query text is converted into a dense vector using the same embedding model that was used for the documents.

3. **Vector DB Search**
   The query vector is compared against the stored document vectors. The top-k most similar chunks are retrieved (often with metadata filtering).

4. **Prompt Construction (Top-k Chunks + Original Query)**
   The retrieved chunks are inserted into a carefully engineered prompt together with the original user question. This is where prompt engineering becomes critical.

5. **LLM Generation**
   The LLM synthesizes an answer grounded in the provided context. Because the relevant facts are in the prompt, hallucinations are reduced and answers can cite sources.

Why RAG works so well

- The LLM never has to "remember" every fact; it only needs to reason over the pieces of evidence you retrieve for it.
- Knowledge can be updated simply by adding or deleting documents in the vector store — no re-training.
- You gain transparency: you can show the source chunks that were used to generate the answer.

> **Key Takeaway**
> RAG quality is determined more by retrieval quality (chunking strategy, embedding model, top-k, re-ranking) than by the size of the LLM. Invest heavily in the retrieval side.

8. Real-World Use Cases & Future Directions

- **Chat with Docs** — Upload internal PDFs, Confluence pages, Notion spaces, or code repositories and let employees ask questions in natural language.
- **Semantic Search** — Replace or augment classic keyword search with understanding of intent.
- **AI Agents** — Agents use vector stores as long-term memory and as a tool for looking up information mid-reasoning.
- **Recommendations** — User behaviour, content, and product descriptions are embedded for hyper-relevant suggestions.
- **Legal AI** — Contract review, discovery, and case-law research.
- **Education** — Personalized tutoring systems that retrieve the exact explanation or exercise a student needs.

Looking ahead: hybrid search, multi-modal embeddings (text + image + audio), graph-enhanced RAG, and agentic retrieval are the active frontiers.
